

Comparative Analysis of Gemini 3.1 Pro, Claude Opus 4.6, and GPT-OSS in Vibe-Coding

Burak Demirci

2201292

Artificial Intelligence Engineering
Bahçeşehir University

Abstract—This paper examines the performance of Gemini 3.1 Pro, Claude Opus 4.6, and GPT-OSS 120b in the context of AI-assisted coding. The study focuses on how these models handle code generation and understanding through natural language instructions. Additionally, the paper describes a workflow using AI-based research tools to assist in academic writing. The results provide a direct comparison of the strengths and weaknesses of each model in a practical development environment.

Index Terms—LLMs, Gemini 3.1 Pro, Claude Opus 4.6, GPT-OSS 120b, vibe-coding, software development, comparison

I. INTRODUCTION

AI-assisted coding, popularly known as 'vibe-coding', has shifted software development from manual syntax writing to natural language intent. Popularized by Andrej Karpathy, this approach treats English as a primary programming tool, allowing developers to guide AI models through a process of conversational co-creation. It redefines the developer as an orchestrator who manages the project's direction through collaborative dialogue rather than strict code implementation [1].

The purpose of this study is to compare three major and cutting-edge AI models: Gemini 3.1 Pro, Claude Opus 4.6, and GPT-OSS 120b. These models were selected to represent a mix of proprietary solutions. The paper evaluates their accuracy in coding tasks and documents how AI tools can also support the research and writing process for academic papers.

II. LITERATURE REVIEW

The literature review for this study focuses on the official technical documentations and model cards provided by the developers of the selected Large Language Models (LLMs). Since these models are at the forefront of the generative AI industry, their technical reports serve as the primary source for benchmarking their coding and reasoning capabilities.

The analysis primarily utilizes the Gemini 3.1 Pro Technical Report from Google, the Claude 4.6 Performance Report from Anthropic, and the GPT-OSS 120b Model Card published on arXiv. These documents provide standardized benchmark results that allow for a direct comparison of the models' performance in software engineering tasks. A key focus of current literature is the SWE-bench (Software Engineering Benchmark), which evaluates a model's ability to resolve real-world GitHub issues. While proprietary models like Gemini and Claude report high success rates (exceeding 80%) [2] [3],

the open-source community provides alternatives like GPT-OSS. Although GPT-OSS shows high efficiency in reasoning tasks, its reported SWE-bench score of 62.4% [4] suggests a performance gap in complex system integrations compared to its proprietary counterparts.

In addition to coding models, recent literature emphasizes the importance of AI-driven research assistants. Tools such as Elicit and SciSpace are documented to improve the efficiency of literature synthesis and data extraction from academic papers. This study integrates these tools to streamline the documentation process, ensuring that the paper adheres to academic standards while utilizing modern AI capabilities.

III. OVERVIEW OF EXISTING AI CODING SYSTEMS

The current ecosystem of AI-assisted coding can be divided into two primary categories based on their integration and deployment:

A. General-Purpose Proprietary LLMs

These are high-capacity models developed by major technology firms, accessible via web interfaces or APIs. Models like **Gemini 3.1 Pro** and **Claude Opus 4.6** fall into this category. They are characterized by massive context windows and advanced reasoning capabilities, allowing them to handle entire codebases rather than just single functions.

B. Open-Source and Open-Weight Models

To ensure privacy and transparency, the open-source community has developed models like **Llama-3**, **DeepSeek-V3**, and **GPT-OSS 120b**. These models allow developers to run powerful AI systems on local hardware or private servers. While they often require more optimization, they provide a crucial alternative to the "black-box" nature of proprietary systems.

IV. SELECTED TOOLS AND JUSTIFICATION OF SELECTION

The selection of tools for this study was driven by the need for architectural diversity and the inclusion of the latest advancements in Large Language Models as of early 2026. While **Elicit** was utilized as a specialized research assistant to facilitate the discovery and analysis of scholarly literature, all coding models were accessed and tested through the **Antigravity** platform to ensure a unified testing environment and to minimize interface-induced variables during the vibe-coding process.

A. Selection Criteria

The primary criteria for selection included:

- **Aggregator Efficiency:** Using an aggregator like Anti-gravity allowed for seamless switching between models, ensuring that the "vibe" (prompt) remained identical across all tests.
- **Recency:** Only models released or significantly updated within the last six months were considered to reflect the current state of the art.
- **Architectural Diversity:** A mix of proprietary and open-source models was chosen to compare different development philosophies.

B. Justification of Selected Models

The primary criteria for selection included:

- **Gemini 3.1 Pro:** Chosen for its advanced reasoning capabilities and massive context window. It represents the pinnacle of multimodal integration, providing a unique perspective on how deep reasoning affects code generation.
- **Claude Opus 4.6:** Selected as the benchmark for high-end software engineering. Its reputation for logical consistency and "agentic" coding behavior makes it a vital subject for studying complex code synthesis.
- **GPT-OSS 120b:** Included to evaluate the current capabilities of the open-source ecosystem. This model serves as a control group to determine if transparent, community-driven architectures can compete with proprietary giants in a vibe-coding context.

V. EXPERIMENTAL CODING EXAMPLES USING AI TOOLS

To evaluate the practical performance of the selected models, a single, complex "vibe-coding" prompt was used. The prompt required the models to generate a Python-based Timezone Conversion tool with a specific Graphical User Interface (GUI) and logic constraints.

- **Logic:** Time conversion using web scraping for timezone offsets and manual arithmetic.
- **Edge Cases:** Detection of "Next/Previous Day" and error handling for unrelated text.
- **UI Design:** A 1000x500 (2:1 ratio) interface with a specific three-column layout (Input, Button, Output)

The experiment followed a "one-shot" generation approach where each model received the same natural language instruction without prior context.

The whole given prompt were:

*write me a python code for timezone conversion, for inputs such as "6 am PT" return the desired conversion such as "16:00 GMT+X", "12 pm XX" or "4:00 UTC-X", etc.
input and output can be given by user and can be any time zone either UTC, GMT, or any other, output can also be a location, if a location is given first the time zone code will be found then the corresponding time zone conversion will be made,
if nothing is written in output or location is given give time in*

24-hour time format,

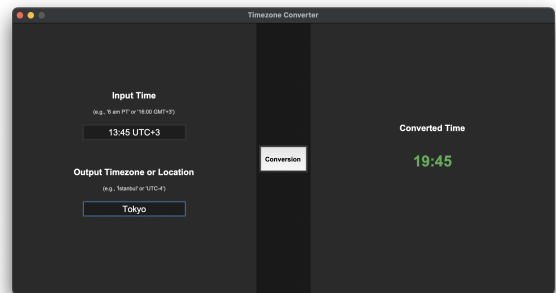
if the output is the next or previous day it should tell in the end of response " — Next/Previous Day"

if any unrelated text is given, system should respond an ERROR in the output area all operations must be made by webscraping methods and arithmetic operations, first find the offset of time zone for either input or output, except if a location is given find the time zone then find the offset of found time zone, then calculate the time for desired time-zone/location, all operations should be managed with a basic interface which would be like 1000x500 (2:1 ratio), where left half (450x500) will be user text-input area, the middle will include "Conversion" button (100x500) and, the right half (450x500) will be output area

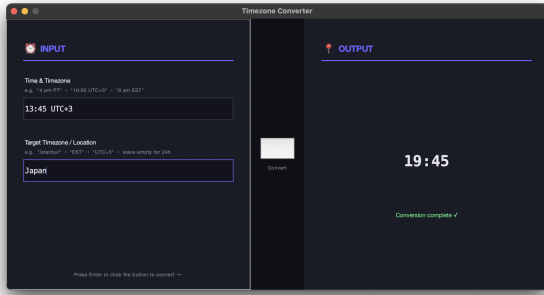
example usage: "4 pm PT" as input and "İstanbul" as output are written then Conversion button is clicked, the python function executes, when finished output area will be updated and shown "2:00 — Next Day"

VI. COMPARISON OF THE TOOLS

- **Gemini 3.1 Pro** delivered a visually satisfying interface; however, it partially deviated from the tripartite layout instructions by placing the output field on the left and creating a secondary display on the right upon conversion. Logically, the model handled timezone offsets and arithmetic operations (including "Next/Previous Day" logic) with high precision. Notably, Gemini adopted an incremental development workflow, requesting step-by-step validation and testing before finalizing the code, which ensured a lower bug rate despite failing the specific "ERROR" requirement for invalid text.



- **Claude Opus 4.6** produced the most aesthetically pleasing GUI among the three models. Similar to Gemini, it misaligned the initial layout but excelled in the execution of web-scraping for offsets. A significant observation was its sensitivity to location granularity; while country-level queries were successful, city-level scraping occasionally resulted in errors. Claude also demonstrated advanced agentic behavior, planning the project structure and insisting on iterative testing phases, which highlights a shift from simple code completion to interactive software engineering.



- **GPT-OSS 120b** was the only model to correctly place the output field on the right side as requested, the overall design was poorly executed, with small fonts and overlapping elements. Functionally, the model failed the core requirements; the web-scraping logic was incomplete, resulting in malformed queries and constant execution errors. Unlike its proprietary counterparts, GPT-OSS attempted to generate the entire codebase in a single pass without iterative testing or confirmation, leading to a non-functional end product.

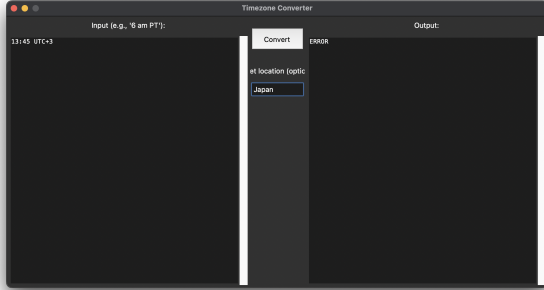


TABLE I
MODEL PERFORMANCE ACROSS FUNCTIONAL AND DESIGN METRICS

Metric	Gemini 3.1	Opus 4.6	GPT-OSS
GUI Size (1000x500)	Correct	Correct	Correct
Layout Accuracy	Medium	Medium	High
Visual Aesthetics	High	Very High	Very Low
Scraping (Offsets)	Success	Success	Failed
Scraping (Location)	Partial	Partial	Failed
Error Handling	Failed	Failed	Success†
Functional Status	Operational	Operational	Broken

† GPT-OSS returned errors for all inputs, failing to distinguish between unrelated text and valid queries.

VII. DISCUSSION

The findings of this study highlight that while "vibe-coding" enables rapid prototyping, a notable gap remains between natural language intent and architectural precision. The proprietary models, Gemini 3.1 Pro and Claude Opus 4.6, demonstrated superior performance by adopting an incremental, "agentic" workflow, insisting on step-by-step validation. In contrast, the single-pass generation attempt of GPT-OSS 120b resulted in a non-functional product, suggesting that the ability to plan and self-correct is more vital for complex software tasks than raw parameter count. Furthermore, while AI-supported research tools like SciSpace and Elicit significantly streamline the documentation process, human oversight remains essential for critical synthesis and technical accuracy. Ultimately, these results indicate that the developer's role is shifting from manual syntax writing to high-level orchestration and auditing of AI-generated outputs.

VIII. CONCLUSION

This study concludes that while "vibe-coding" represents a significant shift toward intent-based software development, the efficacy of the process is currently dominated by proprietary models with advanced agentic reasoning. Gemini 3.1 Pro and Claude Opus 4.6 proved that an iterative, step-by-step development workflow is essential for successfully integrating complex features like GUI design and web-scraping. In contrast, the failure of GPT-OSS 120b to produce functional code underscores the remaining performance gap between open-source and proprietary architectures in multi-modular tasks. Additionally, the use of AI-supported research tools was found to enhance academic productivity, provided that human oversight is maintained for technical validation. Ultimately, AI-assisted tools are not a replacement for engineering expertise but a powerful augmentation that redefines the developer as an orchestrator of intelligent systems.

REFERENCES

- [1] V. Pimenova, S. Fakhoury, C. Bird, and T. Zimmermann, "Good Vibrations? A Qualitative Study of Co-Creation, Communication, Flow, and Trust in Vibe Coding," *arXiv preprint arXiv:2509.12491*, 2025.
- [2] Google, "Introducing Gemini 3.1 Pro: Our most capable model for reasoning and coding," *Google Blog*, 2026. [Online]. Available: LINK
- [3] Anthropic, "Claude 4.6 Model Card and Technical Report," *Anthropic Technical Assets*, 2026. [Online]. Available: LINK
- [4] GPT-OSS Team, "gpt-oss-120b & gpt-oss-20b Model Card," *arXiv preprint arXiv:2508.10925*, 2025. [Online]. Available: LINK